ChartNav — Consolidated Build

Backend phases 1–5: spine → state machine → auth → RBAC → CI/hardening
Repo: `thekidd2227/chartnav-platform`

Contents

1. Executive summary
2. 01 — Current state
3. 02 — Workflow state machine
4. 03 — API endpoints
5. 04 — Data model
6. 05 — Build log
7. 06 — Known gaps + verification matrix
8. 07 — Auth, org scoping, RBAC
9. 08 — Test strategy
10. 09 — CI & deploy hardening
11. 10 — Doc artifact pipeline
12. 11 — Production-auth seam
13. 12 — Runtime config
14. 13 — Deploy target
15. 14 — Postgres parity
16. Diagram — System architecture
17. Diagram — Encounter status machine
18. Diagram — ER
19. Diagram — API / data flow

Executive summary

This document consolidates **all** ChartNav backend phases delivered to

date.

Phase 1 — Workflow spine. Alembic migration `a1b2c3d4e5f6` introduced `encounters` and `workflow_events` tables with FKs and indexes. Six encounter endpoints. Idempotent seed.

Phase 2 — State machine + filtering. `POST /encounters/{id}/status` enforces explicit forward + rework edges. `GET /encounters` accepts `organization_id`, `location_id`, `status`, `provider_name` filters.

Phase 3 — Dev auth + org scoping. `apps/api/app/auth.py` resolves the caller from `X-User-Email`. Every encounter route became org-scoped: cross-org reads return 404, cross-org body/query assertions 403.

Phase 4 — RBAC + full scoping + tests. `apps/api/app/authz.py` introduces `admin` / `clinician` / `reviewer` roles and per-edge transition rules. `/organizations`, `/locations`, `/users` are now authenticated and scoped.

`{error_code, reason}` envelope is standardized. 25-test pytest suite covers auth, scoping, RBAC, state-machine invariants.

Phase 5 — CI + runtime hardening. GitHub Actions workflow runs install + Alembic upgrade on an isolated CI DB + idempotent seed + pytest + live smoke. A separate `docs` job regenerates the final HTML/PDF. Root `Makefile`, `apps/api/scripts/smoke.sh`, and `scripts/build_docs.py` codify the local verification + doc-build paths.

Phase 6 — Production seam + deploy target + Postgres parity (this phase). DB layer moved to SQLAlchemy Core (`apps/api/app/db.py`) — same queries run on SQLite and Postgres via `:name` binds, `COALESCE`, and an `insert_returning_id` helper. New `apps/api/app/config.py` centralizes env-driven settings and validates auth mode at import time. `auth.py` gained a `resolve_caller_from_bearer` placeholder that returns **501** rather than fake-serving traffic. Dockerfile hardened (non-root, HEALTHCHECK, entrypoint that runs migrations at start). `infra/docker/docker-compose.prod.yml` brings up API + Postgres. `scripts/pg_verify.sh` spins a throwaway Postgres container and proves migrations + seed + boot + smoke + a real state transition + event write. CI expanded to four jobs: `backend-sqlite`, `backend-postgres`, `docker-build`, `docs`.

Preserved untouched: `/health`, `/`, Alembic history (still 2 revisions), SQLite dev workflow, state machine + filtering surface, workflow_events model, existing endpoint contracts.

01 — Current state

ChartNav — Current State

As of: 2026-04-18 (phase: production seam, deploy target, Postgres parity)

Repo layout (relevant)

```
chartnav-platform/
├── .github/workflows/ci.yml         # backend-sqlite + backend-postgres + docker-build + docs
├── Makefile                         # install · migrate · seed · test · verify · pg-verify · docker-*
├── scripts/
│   ├── build_docs.py                # reproducible HTML/PDF build
│   ├── verify.sh                    # boot + smoke teardown
│   └── pg_verify.sh                 # NEW — end-to-end Postgres parity proof
├── apps/
│   ├── api/
│   │   ├── Dockerfile               # hardened: non-root, healthcheck, entrypoint
│   │   ├── entrypoint.sh            # NEW — migrate-on-start, optional seed, exec CMD
│   │   ├── .env.example             # runtime contract (env-first)
│   │   ├── app/main.py
│   │   ├── app/config.py            # NEW — central settings from env
│   │   ├── app/db.py                # NEW — SA Core, cross-dialect
│   │   ├── app/auth.py              # header + bearer resolvers behind one seam
│   │   ├── app/authz.py             # RBAC
│   │   ├── app/api/routes.py        # all queries now :name-bound
│   │   ├── alembic.ini
│   │   ├── alembic/env.py           # honors `-x` AND `DATABASE_URL`
│   │   ├── alembic/versions/        # 2 migrations (unchanged)
│   │   ├── scripts_seed.py          # rewritten on SA; cross-dialect
│   │   ├── scripts/smoke.sh         # shell smoke (reused across envs)
│   │   └── tests/
│   │       ├── conftest.py          # env-based per-test DB
│   │       ├── test_auth.py
│   │       ├── test_auth_modes.py   # NEW — header + bearer seam
│   │       ├── test_rbac.py
│   │       └── test_scoping.py
│   └── web/
├── infra/docker/
│   ├── docker-compose.yml           # dev stack
│   └── docker-compose.prod.yml      # NEW — API + Postgres
└── docs/
    ├── build/ 01 … 14
    ├── diagrams/
    └── final/*.html / *.pdf
```

Runtime baseline

- Python 3.11+, FastAPI, **SQLAlchemy Core** now underpins all DB access.

- Database: `DATABASE_URL` → `sqlite:///apps/api/chartnav.db` (dev default) or `postgresql+psycopg://…` (prod / parity CI).
- Alembic head: `a1b2c3d4e5f6`. No new migrations this phase.
- Auth: `CHARTNAV_AUTH_MODE` (`header` dev / `bearer` prod-shaped placeholder). Config refuses to import if `bearer` is set without JWT env.
- RBAC: `admin` / `clinician` / `reviewer`.
- Error envelope: `{"detail": {"error_code": "...", "reason": "..."}}`.
- Deploy target: Docker image + `docker-compose.prod.yml` (API + Postgres).
- Tests: 28 pytest on SQLite (incl. 3 new auth-mode tests); Postgres parity proved end-to-end via `scripts/pg_verify.sh` and a dedicated CI job.

Verified working endpoints

Unchanged since phase 4. Now verified on BOTH SQLite and Postgres where applicable:

- Open: `GET /health`, `GET /`
- Authed: `GET /me`, `GET /organizations`, `GET /locations`, `GET /users`, `GET /encounters` (+ filters), `GET /encounters/{id}`, `GET /encounters/{id}/events`
- Authed + RBAC: `POST /encounters` (admin, clinician), `POST /encounters/{id}/events` (admin, clinician), `POST /encounters/{id}/status` (per-edge roles)

Automation

- `make verify` — SQLite: reset-db + pytest (28) + boot + smoke (9) + teardown.
- `make pg-verify` — spins throwaway Postgres, migrates + seeds + boots + smokes + exercises state transition. Teardown trap-guaranteed.
- `make docker-build` / `make docker-up` / `make docker-down`.
- `python scripts/build_docs.py` — deterministic HTML + PDF.
- CI: `backend-sqlite` → `backend-postgres` + `docker-build` + `docs` (each gated on its predecessor where it matters).

Encounter Workflow State Machine

Status values

Status	Meaning
`scheduled`	Visit is booked, patient has not arrived.
`in_progress`	Provider is seeing the patient / actively charting.
`draft_ready`	Note draft has been generated and is awaiting review.
`review_needed`	Reviewer flagged the note — needs revision or sign-off.
`completed`	Note signed off; encounter closed. **Terminal.**

Allowed transitions

Forward flow:

```
scheduled → in_progress → draft_ready → review_needed → completed
```

Explicit rework transitions (documented + enforced):

From	To	Purpose
`draft_ready`	`in_progress`	Draft rejected, return to charting.
`review_needed`	`draft_ready`	Reviewer kicks note back for rewrite.

Rejected transitions (examples that return HTTP 400)

- `scheduled → completed`
- `scheduled → review_needed`
- `scheduled → draft_ready`
- `in_progress → scheduled`
- `in_progress → completed` (must pass through draft_ready + review_needed)
- `completed → anything` (terminal)
- Any unknown status string (`invalid_status`)

Error body format:

```
{"detail":"invalid_transition: in_progress -> completed is not permitted; allowed next states from in_
```

Idempotency

Posting the **same** status the encounter is already in is a no-op: the row

is returned unchanged and **no** `status_changed` event is appended.

Side effects on successful transition

Transition target	Timestamp action
`in_progress`	set `started_at = now()` if currently NULL
`completed`	set `completed_at = now()` ; also set `started_at = now()` if it was NULL
other	no timestamp mutation

Every successful transition appends a `workflow_events` row:

```
{
  "event_type": "status_changed",
  "event_data": {"old_status": "...", "new_status": "..."}
}
```

Diagram

See `docs/diagrams/encounter-status-machine.md` (Mermaid).

Source of truth

`apps/api/app/api/routes.py` :

- `ALLOWED_STATUSES` — the 5 permitted status strings.
- `ALLOWED_TRANSITIONS` — `dict[str, set[str]]` , one entry per source state.
- `update_encounter_status()` — the single enforcement point.

API Endpoints

Base URL (local dev): `http://127.0.0.1:8000` . All error bodies use

`{"detail": {"error_code": "...", "reason": "..."}}` .

System (open)

Method	Path
GET	`/health`
GET	`/`

Identity

Method	Path	Auth	Roles
GET	`/me`	🔒	any

Org metadata (🔒, org-scoped)

Method	Path	Behavior
GET	`/organizations`	Returns only caller's org row.
GET	`/locations`	`WHERE organization_id = caller.org` .
GET	`/users`	`WHERE organization_id = caller.org` .

Encounters (🔒, org-scoped + RBAC)

GET `/encounters`

Any authenticated role. Always `WHERE organization_id = caller.org` .

Optional query params (AND-ed, parameterized):

Param	Rule
`organization_id`	Must equal `caller.org` — else 403.
`location_id`	Narrows within caller org.

Param	Rule
`status`	Must be in `ALLOWED_STATUSES` — else 400.
`provider_name`	Exact match.

GET `/encounters/{id}` — any role

404 if missing or cross-org.

GET `/encounters/{id}/events` — any role

404 if parent missing or cross-org.

POST `/encounters` — admin, clinician

Body:

```
{
  "organization_id": 1,
  "location_id": 1,
  "patient_identifier": "PT-1003",
  "patient_name": "Alex Chen",
  "provider_name": "Dr. Ortiz",
  "scheduled_at": "2026-04-18T14:30:00Z",
  "status": "scheduled"
}
```

POST `/encounters/{id}/events` — admin, clinician

Reviewer → 403. 404 if cross-org.

POST `/encounters/{id}/status`

Per-edge RBAC:

From	To	Roles allowed
scheduled	in_progress	admin, clinician
in_progress	draft_ready	admin, clinician
draft_ready	in_progress	admin, clinician
draft_ready	review_needed	admin, reviewer
review_needed	draft_ready	admin, reviewer
review_needed	completed	admin, reviewer

Same-state POST = no-op (200). Role violation → 403 `role_cannot_transition`. Invalid transition → 400 `invalid_transition`. Cross-org → 404.

Error code inventory

Code	HTTP	Origin
`missing_auth_header`	401	auth transport
`unknown_user`	401	auth transport
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to unknown
`role_forbidden`	403	generic `require_roles`
`role_cannot_create_encounter`	403	POST /encounters
`role_cannot_create_event`	403	POST /encounters/{id}/events
`role_cannot_transition`	403	POST /encounters/{id}/status
`cross_org_access_forbidden`	403	body/query asserts other org
`encounter_not_found`	404	missing or cross-org encounter
`location_not_found`	400	bad location_id
`invalid_status`	400	unknown status string
`invalid_initial_status`	400	POST /encounters bad init
`invalid_transition`	400	disallowed state-machine edge

Verification

This full surface is now locked in by two layers:

- pytest (`apps/api/tests/`) — 25 tests, per-test ephemeral SQLite.
- Live smoke (`apps/api/scripts/smoke.sh`) — 9 assertions against a running API.

Both run in CI on every push/PR. See `09-ci-and-deploy-hardening.md` .

04 — Data model

Data Model

SQLite (local dev). Schema produced by Alembic migrations

`43ccbf363a8f → a1b2c3d4e5f6` . No schema changes in this RBAC/scoping phase.

Tables (unchanged this phase)

`organizations`

column	type	constraints
id	INTEGER	PK
name	VARCHAR(255)	NOT NULL
slug	VARCHAR(255)	NOT NULL, UNIQUE
created_at	DATETIME	NOT NULL default now()

`locations`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
name	VARCHAR	NOT NULL
created_at	DATETIME	NOT NULL default now()

`users`

column	type	constraints
id	INTEGER	PK
organization_id	INTEGER	NOT NULL, FK → organizations(id)
email	VARCHAR	NOT NULL, UNIQUE
full_name	VARCHAR	NULL
role	VARCHAR	NOT NULL default `"admin"`
created_at	DATETIME	NOT NULL default now()

Application-level role vocabulary (enforced by `app/authz.py`):

`admin` , `clinician` , `reviewer` . The column remains a free VARCHAR in

SQL; constraint is enforced at the app layer. If you need a DB-level

CHECK constraint, add it in a future migration.

`encounters` and `workflow_events`

Unchanged.

Seeded tenants (two, with full role coverage)

Org 1 — `demo-eye-clinic` (id=1)

Location: `Main Clinic` (id=1).

email	role
admin@chartnav.local	admin
clin@chartnav.local	clinician
rev@chartnav.local	reviewer

Encounters:

id	patient	provider	status	events
1	PT-1001	Dr. Carter	`in_progress`	3
2	PT-1002	Dr. Patel	`review_needed`	5

Org 2 — `northside-retina` (id=2)

Location: `Northside HQ` (id=2).

email	role
admin@northside.local	admin
clin@northside.local	clinician

Encounters:

id	patient	provider	status	events
3	PT-2001	Dr. Ahmed	`scheduled`	1

Relationships

Unchanged — see `docs/diagrams/er-diagram.md`.

Event taxonomy

event_type	Writer	Shape
`encounter_created`	`POST /encounters` / seed	`{"status": "...", "created_by": "<email>"}`
`status_changed`	`POST /encounters/{id}/status`	`{"old_status": "...", "new_status": "...", "changed_by": "<email>"}`
user-supplied	`POST /encounters/{id}/events`	Any JSON

05 — Build log

Build Log

Reverse-chronological.

2026-04-18 — Phase 6: production seam + deploy target + Postgres parity

Step 1 — Baseline

- Starting head: `cfa8ca9` (CI + runtime hardening).

Step 2 — Runtime config module

- New `apps/api/app/config.py`. All env reads centralized into `settings = _load()`. Validates `CHARTNAV_AUTH_MODE` and demands the three `CHARTNAV_JWT_*` vars if mode is `bearer` — otherwise raises at import time. No module besides `app.config` touches `os.environ`.
- New `apps/api/.env.example` with the full documented contract.

Step 3 — DB layer (SQLAlchemy Core, cross-dialect)

- New `apps/api/app/db.py` — `engine`, `transaction()`, `fetch_all`, `fetch_one`, `insert_returning_id`. SQLite connect-args and PRAGMA FK enforcement handled automatically.
- Refactored `apps/api/app/api/routes.py`: every query now uses named (`:name`) bind parameters and `transaction()` for writes. The old `sqlite3`-specific `lastrowid` is gone.
- Refactored `apps/api/scripts_seed.py` to the same shape. Swapped `IFNULL` → `COALESCE`. Timestamp columns still use server-side `CURRENT_TIMESTAMP` for dialect-agnostic behavior.
- `apps/api/app/auth.py` now reads users via `db.fetch_one`; no more direct sqlite3 use.
- `apps/api/alembic/env.py` now also honors `DATABASE_URL` from env (already honored `-x sqlalchemy.url=`), so Alembic works uniformly in

CI and the deploy entrypoint.

Step 4 — Auth seam hardening

- Split resolvers: `resolve_caller_from_header` (dev) and

`resolve_caller_from_bearer` (prod-shaped placeholder). `require_caller`

dispatches via `settings.auth_mode`.

- Bearer mode returns **501** `auth_bearer_not_implemented` when a

token is actually presented. That's deliberate — it's better to fail

loudly than pretend half-auth.

- Added `apps/api/tests/test_auth_modes.py` (3 tests) asserting:
- bearer-mode without JWT env → `RuntimeError` at `import app.config`
- bearer-mode with JWT env but a token → 501
- bearer-mode without a token → 401 `missing_auth_header`
- header-mode (default) still returns correct caller

Step 5 — Deploy target

- `apps/api/Dockerfile` rewritten: non-root user, `curl`-based

HEALTHCHECK, `entrypoint.sh`, installs the `[postgres]` extra.

- `apps/api/entrypoint.sh` NEW: asserts `DATABASE_URL`, runs

`alembic upgrade head`, optionally runs seed when

`CHARTNAV_RUN_SEED=1`, then `exec`s uvicorn.

- `infra/docker/docker-compose.prod.yml` NEW: API + Postgres with a

healthcheck dependency gate and env-driven defaults.

Step 6 — Postgres parity

- `apps/api/pyproject.toml` gains a `[postgres]` extra pinning

`psycopg[binary]>=3.2`.

- `scripts/pg_verify.sh` NEW: throwaway `postgres:16-alpine` container

→ migrate → seed twice (idempotency) → boot API → run the shared

smoke script → live status transition → confirm `workflow_events`

row was written. Traps cleanup on any exit.

- Local run: **PASS** on 2026-04-18.

Step 7 — CI expansion

- `.github/workflows/ci.yml` now has four jobs:

- `backend-sqlite` — existing pytest + SQLite smoke (unchanged contract).
- `backend-postgres` — service-container Postgres, alembic upgrade,

seed twice, boot, smoke, live status transition.

- `docker-build` — buildx build of the API image, run it with

SQLite + `CHARTNAV_RUN_SEED=1`, smoke the live container.

- `docs` — regenerate HTML + PDF, upload as artifact.
- `docker-build` and `docs` both `needs: backend-sqlite`;

`backend-postgres` also `needs: backend-sqlite`.

Step 8 — Makefile + tooling

- New targets: `pg-verify`, `docker-build`, `docker-up`, `docker-down`.
- `verify.sh` already tolerant of missing `.venv/bin/uvicorn` (falls

back to PATH) so both local and CI paths work.

Step 9 — Tests updated

- `apps/api/tests/conftest.py` now sets `DATABASE_URL` via monkeypatch

per test and flushes cached `app.*` modules so `settings` re-reads

env cleanly. Works identically for SQLite (today) and Postgres (a

matrix we'll flip when ready).

- pytest suite: **28 passed** locally and in `backend-sqlite`.

Step 10 — Docs

- New: `11-production-auth-seam.md`, `12-runtime-config.md`,

`13-deploy-target.md`, `14-postgres-parity.md`.

- Updated: `01-current-state.md`, `05-build-log.md`, `06-known-gaps.md`,

`07-auth-and-scoping.md`, `08-test-strategy.md`,

`09-ci-and-deploy-hardening.md`. Diagrams refreshed

(`system-architecture`, `api-data-flow`).

- `scripts/build_docs.py` now picks up `11…14` automatically.
- Final HTML + PDF regenerated.

Step 11 — Hygiene

- Dev DB reset to seeded state before commit.
- `.gitignore` already excludes `.pytest_cache`, `__pycache__`, `*.db`.

Prior phases

- **Phase 5 — CI + runtime hardening + doc pipeline** (`cfa8ca9`)
- **Phase 4 — RBAC + full scoping + pytest** (`c6f29e6`)
- **Phase 3 — Dev auth + org scoping** (`efb5b56`)
- **Phase 2 — Strict state machine + filtering** (`505f025`)
- **Phase 1 — Workflow spine** (`93fceb4`)

06 — Known gaps + verification matrix

Known Gaps & Verification Matrix

Verification evidence — phase 6

Local `make verify` (SQLite)

Step	Result
`rm -f apps/api/chartnav.db`	✅
`alembic upgrade head`	✅
`scripts_seed.py` (x2 idempotent)	✅
`pytest tests/ -v`	✅ **28/28**
`uvicorn` boots	✅
`scripts/smoke.sh` 9 assertions	✅
teardown clean	✅

Local `make pg-verify` (Postgres 16)

Step	Result
throwaway `postgres:16-alpine` comes up	✅
`alembic upgrade head` on Postgres	✅ both revisions applied
seed on Postgres (run twice, idempotent)	✅
uvicorn boots against Postgres	✅
`scripts/smoke.sh` 9 assertions	✅
clinician status transition `in_progress → draft_ready`	✅
`workflow_events` row written with `old_status` / `new_status` / `changed_by`	✅
container torn down (trap)	✅

Auth seam tests (`tests/test_auth_modes.py`)

Case	Result
`CHARTNAV_AUTH_MODE=bearer` w/o JWT env → RuntimeError at import	✅
bearer with JWT env, no token → 401 `missing_auth_header`	✅
bearer with JWT env, token present → 501 `auth_bearer_not_implemented`	✅

Case	Result
header mode default → 200 + correct role/org	✅

Docker build

- `docker build -t chartnav-api:local apps/api` — runs locally.
- CI `docker-build` job builds with buildx and smokes the live

container — enforced per PR.

Real gaps (prioritized for next phase)

1. **JWT/SSO validation is still a stub.** Bearer mode rejects all traffic with 501. Next phase: PyJWT + JWKS fetch/cache, issuer/audience validation, claim → user mapping.

2. **pytest runs on SQLite only.** Postgres parity is asserted by `pg_verify.sh` + the `backend-postgres` CI job (which exercises the live surface). A pytest Postgres matrix is the next CI upgrade; `conftest.py` already reads env so this is mostly wiring.

3. **No image push / release pipeline.** CI builds the Docker image but doesn't publish it.

4. **Secrets are still plain env vars.** No AWS SM / Vault integration.

5. **No RBAC-gated writes for org metadata** (`/organizations` etc. remain read-only).

6. **`users.role` is still free VARCHAR** at DB level; no CHECK/lookup.

7. **No pagination / cursor** on `GET /encounters` .

8. **No encounter update / delete / cancel**; no `cancelled` status.

9. **Free-form `event_data`** — no per-event_type schema.

10. **CORS `allow_origins=["*"]` ** still open.

11. **No distinct audit log** for auth/scoping failures.

12. **No rate limiting, no lockout, no structured logging.**

Auth, Org Scoping, RBAC

1. Authentication (authn)

Source: `apps/api/app/auth.py`.

- Transport today: request header `X-User-Email: <email>`.
- `require_caller` is a FastAPI dependency that resolves the user from

the `users` table and returns a `Caller(user_id, email, full_name, role, organization_id)`.

- Environment flag `CHARTNAV_AUTH_MODE` (default `"header"`) is the

production upgrade seam. When JWT/SSO lands, only the body of

`require_caller` (or a new branch in it) changes. Every route and

every RBAC helper continues to work unchanged because they depend on

`Caller`, not on any header.

Dev-only, explicitly

`X-User-Email` is trivially spoofable. This is acceptable for local

development only. **Do not deploy this to a shared environment

without first swapping the transport.** The seam is there; use it.

Standardized error envelope

Every auth/authz error returns:

```
{"detail": {"error_code": "<stable_code>", "reason": "<human message>"}}
```

Authn error codes

Code	HTTP	When
`missing_auth_header`	401	`X-User-Email` absent or empty.
`unknown_user`	401	Email not found in `users`.
`auth_mode_unsupported`	500	`CHARTNAV_AUTH_MODE` set to something unwired.

2. Authorization (authz / RBAC)

Source: `apps/api/app/authz.py`.

Roles

Role	Intent
`admin`	Full read/write inside own org.
`clinician`	Charting side of the workflow.
`reviewer`	Review side of the workflow; read-only on create/events.

All seeded users today carry exactly one of these roles.

Permission surface

Surface	admin	clinician	reviewer
Read org / locations / users	✓	✓	✓
List / read encounters	✓	✓	✓
Read encounter events	✓	✓	✓
Create encounter	✓	✓	×
Add workflow event	✓	✓	×
Transition scheduled→in_progress	✓	✓	×
Transition in_progress→draft_ready	✓	✓	×
Transition draft_ready→in_progress (rework)	✓	✓	×
Transition draft_ready→review_needed	✓	×	✓
Transition review_needed→draft_ready (kick back)	✓	×	✓
Transition review_needed→completed	✓	×	✓

Authz dependencies

- `require_roles(*roles)` — generic gate, returns `Caller`.
- `require_create_encounter` — applied to `POST /encounters`.
- `require_create_event` — applied to `POST /encounters/{id}/events`.
- `assert_can_transition(caller, from, to)` — called by the status

handler **after** the state machine accepts the edge.

Authz error codes

Code	HTTP	When
`role_forbidden`	403	Generic `require_roles` denies.
`role_cannot_create_encounter`	403	Non-admin/clinician POSTs `/encounters`.
`role_cannot_create_event`	403	Non-admin/clinician POSTs event.

Code	HTTP	When
`role_cannot_transition`	403	Role may not drive that specific state edge.

3. Org scoping

Caller's `organization_id` is **authoritative**. Bodies and query

strings cannot override it.

Route	Scoping
`GET /organizations`	`WHERE id = caller.org`
`GET /locations`	`WHERE organization_id = caller.org`
`GET /users`	`WHERE organization_id = caller.org`
`GET /encounters`	`WHERE organization_id = caller.org` + filters
`GET /encounters/{id}`	404 if row is cross-org
`GET /encounters/{id}/events`	404 if parent cross-org
`POST /encounters`	Body `organization_id` must equal caller's; location must belong
`POST /encounters/{id}/events`	404 if cross-org
`POST /encounters/{id}/status`	404 if cross-org

404 vs 403

- **404** when the target might-or-might-not exist in another org —

returning 403 there would leak existence.

- **403** when the client explicitly asserts a different org (body or

`?organization_id=`) — the intent is unambiguous, fail loudly.

4. Diagram

5. What this phase explicitly does NOT do

- No password auth, JWT, or SSO integration.
- No scoped writes to organizations / locations / users (meta admin).
- No audit log distinct from `workflow_events`.
- No rate limiting.
- No per-tenant key management.

These are the next auth/security phase's targets.

Test & Smoke Strategy

Stack

- `pytest` + FastAPI `TestClient` for unit/integration.
- Per-test temporary SQLite (fresh migrate + seed per test); env-based wiring so the same fixture will later work against Postgres.
- Live shell smoke (`apps/api/scripts/smoke.sh`) used by:
- `make verify` (SQLite)
- `scripts/pg_verify.sh` (Postgres)
- the built Docker image smoke job
- CI runs all of the above on every push / PR.

Layout

```
apps/api/tests/
├── conftest.py          # env-based per-test DB, header presets
├── test_auth.py         # authn surface (5)
├── test_auth_modes.py   # header + bearer seam (3)
├── test_rbac.py         # role-gated writes + state transitions (12)
└── test_scoping.py      # org scoping (8)

apps/api/scripts/
└── smoke.sh             # curl-level live smoke (9 checks)

scripts/
├── verify.sh            # boot + smoke + teardown (SQLite)
└── pg_verify.sh         # same flow against a throwaway Postgres
```

Coverage

Surface	pytest	smoke	Postgres smoke	Docker-image smoke
`/health` , `/`	✓	✓	✓	✓
Header-mode auth	✓	✓	✓	✓
Bearer-mode seam (501 / config)	✓			
Org scoping	✓	✓	✓	✓
RBAC	✓			
Filter on `/encounters`	✓			

Surface	pytest	smoke	Postgres smoke	Docker-image smoke
State machine invariants	✓		✓	
Event provenance (`changed_by`)	✓		✓	
Migrations			✓	✓
Seed idempotency (x2 runs)			✓	

How to run

```
# Fast pytest (SQLite)
cd apps/api && pytest tests/ -v

# Full local SQLite gate
make verify                 # resets DB, pytest, boot, smoke, teardown

# Postgres parity (docker required)
make pg-verify

# Docker image + smoke
make docker-build
```

Results at time of writing

- `pytest` : **28 passed in ~19s**
- `make verify` (SQLite): 28 pytest + 9 smoke — all green
- `scripts/pg_verify.sh` : migrations + seed (x2) + smoke + status transition + event write — **PASS**

Gaps not yet covered

- pytest matrix against Postgres (the fixture is already env-driven).
- Concurrent writers / race conditions.
- `event_data` schema per `event_type` .
- Pagination behavior (not yet implemented).
- Bearer-mode JWT validation (placeholder only; see `11-production-auth-seam.md`).

CI & Deploy Hardening

GitHub Actions

File: `.github/workflows/ci.yml`. Triggers on **push to `main`** and **every pull request**.

`backend-sqlite`

Install deps → `alembic upgrade head` against an isolated `$RUNNER_TEMP/chartnav_ci.db` via `DATABASE_URL` → seed **twice** (idempotency proof) → `pytest tests/ -v` (28 tests) → `bash scripts/verify.sh` (boots uvicorn, runs `scripts/smoke.sh`).

`backend-postgres` (needs: `backend-sqlite`)

Service container `postgres:16-alpine` on localhost:5432. Install deps (incl. `[postgres]` extra) → `alembic upgrade head` on Postgres → seed twice → boot uvicorn → `scripts/smoke.sh` → live status transition asserts the returned row. Fails the workflow if Postgres parity breaks.

`docker-build` (needs: `backend-sqlite`)

Buildx → build `chartnav-api:ci` from `apps/api/` → run the container with `DATABASE_URL=sqlite:///./chartnav.db` and `CHARTNAV_RUN_SEED=1` → poll `/health` → run `scripts/smoke.sh` against the live container. Proves the production image boots end-to-end.

`docs` (needs: `backend-sqlite`)

apt-install Chromium → `python scripts/build_docs.py` with `CHARTNAV_PDF_BROWSER=chromium-browser` → upload HTML + PDF as `chartnav-docs-final` artifact (`if-no-files-found: error`).

What CI catches

Change	Failure surface
Broken migration	alembic step (both DBs)
Broken seed / re-seed collision	seed step (both DBs)
Any pytest regression	`backend-sqlite` pytest
`/health` or auth drift	`scripts/smoke.sh` (all envs)
Cross-org / state-machine regression	smoke + `backend-postgres` transition
Postgres-only SQL dialect break	`backend-postgres`
Image won't build or boot	`docker-build`
Docs pipeline regression	`docs` job

Local verification

```
make verify        # SQLite end-to-end
make pg-verify     # Postgres end-to-end (Docker required)
make docker-build  # build the production image
make docker-up     # full API + Postgres stack (compose)
```

Individual recipes: `install	migrate	seed	test	boot	smoke	docs	reset-db	clean`.

Dev / test / CI / prod DB separation

Surface	Path
Developer local	`apps/api/chartnav.db` (gitignored)
pytest per-test	`tmp_path / "chartnav.db"` (ephemeral)
CI (sqlite)	`$RUNNER_TEMP/chartnav_ci.db` (ephemeral)
CI (postgres)	`postgres:16-alpine` service container
Local pg-verify	throwaway `chartnav-pg-verify` container on port 55432
Prod compose	`chartnav_pgdata` persistent volume

Smoke script (shared contract)

`apps/api/scripts/smoke.sh [BASE_URL]` — 9 curl assertions:

`/health` (code + body), `/me` (401/200), `/encounters` (401/200), cross-org lens 403, own 200, cross-org 404. Exits non-zero on first failure. Used by **every** smoke path (local `make verify`, Postgres parity, Docker image CI).

What this phase adds beyond CI

- Hardened `Dockerfile` (non-root, healthcheck).
- `apps/api/entrypoint.sh` for deterministic migrate-on-start.
- `infra/docker/docker-compose.prod.yml` for the API + Postgres stack.
- `scripts/pg_verify.sh` — reproducible Postgres parity proof.

What this phase does NOT do

- No image push / registry release.
- No Kubernetes / Terraform / cloud-specific bits.
- No TLS termination or edge reverse proxy.
- No secret store integration.

Those land in a dedicated hosted-deploy phase. This phase makes sure

the seams are in place to plug any of them in without rewrites.

10 — Doc artifact pipeline

Doc Artifact Pipeline

Purpose

The living docs in `docs/build/*` and `docs/diagrams/*` are the source

of truth. The final consolidated artifacts in `docs/final/` are always

rebuildable from those sources — no hand edits.

Source / output layout

```
docs/
├── build/        # human-maintained markdown — source of truth
├── diagrams/     # Mermaid sources (fenced in markdown)
└── final/
    ├── chartnav-workflow-state-machine-build.html   # generated
    └── chartnav-workflow-state-machine-build.pdf    # generated
```

Builder

File: `scripts/build_docs.py`

- Pure Python, no external deps beyond the standard library.
- Resolves paths relative to the script, so the same invocation works

locally and in CI.

- Parses a small markdown subset (headings, fenced code, tables, bullet

lists, inline `code`, **`bold`**) directly to HTML.

- Mermaid fences are passed through verbatim into `<div class="mermaid">`

blocks; the HTML imports Mermaid from a CDN and renders on load.

- Writes `docs/final/chartnav-workflow-state-machine-build.html`.
- Generates the PDF via headless Chromium (`--headless=new --print-to-pdf`).

Selection order:

1. `CHARTNAV_PDF_BROWSER` env var if set (used by CI with `chromium-browser`).

2. macOS Chrome at `/Applications/Google Chrome.app/...`.

3. Chromium in a standard app bundle.

4. `chromium-browser`, `chromium`, `google-chrome`, `chrome` on `PATH`.

- If no browser is available, falls back to `reportlab` for a plain-text

PDF so CI never silently produces a zero-byte artifact.

Run it

Locally:

```
python scripts/build_docs.py          # or:  make docs
```

In CI: the `docs` job in `.github/workflows/ci.yml` installs

`chromium-browser` via apt, runs the builder with

`CHARTNAV_PDF_BROWSER=chromium-browser`, and uploads the resulting

HTML + PDF as a `chartnav-docs-final` artifact. The upload step is

configured with `if-no-files-found: error`, so a silent generation

failure fails the build.

Sections consumed (in order)

1. Executive summary (inlined in the builder)
2. `docs/build/01-current-state.md`
3. `docs/build/02-workflow-state-machine.md`
4. `docs/build/03-api-endpoints.md`
5. `docs/build/04-data-model.md`
6. `docs/build/05-build-log.md`
7. `docs/build/06-known-gaps.md`
8. `docs/build/07-auth-and-scoping.md`
9. `docs/build/08-test-strategy.md`
10. `docs/build/09-ci-and-deploy-hardening.md`
11. `docs/build/10-doc-artifact-pipeline.md`
12. `docs/diagrams/system-architecture.md`
13. `docs/diagrams/encounter-status-machine.md`
14. `docs/diagrams/er-diagram.md`
15. `docs/diagrams/api-data-flow.md`

If a source file is missing, the builder emits an inline `*missing: <path>*` marker rather than crashing — a broken diagram or an undeleted section shows up immediately in the rendered output and in the CI artifact.

Why not a heavier stack

- No MkDocs / Docusaurus / Sphinx — those require a theme + build config we'd have to maintain. We currently just need the 10-page PDF to stay fresh.
- No pandoc dependency — keeps the CI image lean and the local experience `python` + a headless browser (which most devs already have).
- If we grow beyond a single consolidated artifact (e.g. a hosted docs site), this builder is easy to retire because the sources are already plain markdown.

11 — Production-auth seam

Production-Auth Seam

Goal

Make auth transport swappable without touching routes or RBAC.

Where the seam lives

`apps/api/app/auth.py` exposes a single FastAPI dependency:

```
def require_caller(x_user_email: ..., authorization: ...) -> Caller:
    mode = settings.auth_mode
    if mode == "header":  return resolve_caller_from_header(...)
    if mode == "bearer":  return resolve_caller_from_bearer(...)
    raise ...
```

Both resolvers return the **same** `Caller` contract:

```
Caller(user_id, email, full_name, role, organization_id)
```

Every route, every RBAC helper, every test depends on `Caller`, not on a

specific header. Changing transports is a one-file edit inside `auth.py`.

Mode selection

`CHARTNAV_AUTH_MODE` in the environment (see `12-runtime-config.md`):

Value	State	Behavior
`header`	implemented	Reads `X-User-Email`, looks up `users`. Dev only.
`bearer`	placeholder	Requires `Authorization: Bearer …`; returns 501 because JWT validation is not yet wired.

Unknown values cause `app.config` to refuse to import — fail fast.

Bearer mode — honest placeholder

When `CHARTNAV_AUTH_MODE=bearer` is selected, `app.config._load()`

verifies these three values are all present before letting the app come

up at all:

- `CHARTNAV_JWT_ISSUER`
- `CHARTNAV_JWT_AUDIENCE`
- `CHARTNAV_JWT_JWKS_URL`

Missing any one → `RuntimeError` at import time with the list of missing vars. The app will not start with a half-configured bearer setup.

When bearer mode is active and fully configured, `resolve_caller_from_bearer` currently returns:

```
HTTP 501
{"detail": {"error_code": "auth_bearer_not_implemented",
            "reason": "bearer-token validation is not implemented in this phase; set CHARTNAV_AUTH_MOD
```

This is intentional. A future phase will wire real JWT signature verification (PyJWT + JWKS fetch against `CHARTNAV_JWT_JWKS_URL`, issuer/audience checks, claim → `users` lookup). When it ships, only the body of `resolve_caller_from_bearer` changes.

What production-ready vs not yet implemented

Component	State
Mode selection + config	✅ production-shaped; fails fast on gaps
Standardized error envelope	✅ `{error_code, reason}` everywhere
Header transport	✅ works; dev only, do not expose
Bearer transport	⚠ placeholder — returns 501 honestly
JWT signature validation	❌ not implemented
JWKS rotation / cache	❌ not implemented
Token → `users` mapping	❌ not implemented (design TBD)
RBAC integration	✅ works for any transport via `Caller`
Org scoping integration	✅ works for any transport via `Caller`

Tests

`apps/api/tests/test_auth_modes.py`:

- `test_bearer_mode_requires_jwt_env` — setting `CHARTNAV_AUTH_MODE=bearer`

without issuer/audience/JWKS raises at `import app.config`.

- `test_bearer_mode_refuses_to_serve_traffic` — with JWT config set, bearer requests still get a clear 501 `auth_bearer_not_implemented`; missing header returns 401 `missing_auth_header`.
- `test_header_mode_unchanged_contract` — header mode (the default) still resolves seeded users to the expected role/org.

12 — Runtime config

Runtime Configuration

Single source of truth: `apps/api/app/config.py` (`settings`). Every

other module imports from there; nothing else reads `os.environ` .

The contract

Variable	Required?	Default	Purpose
`CHARTNAV_ENV`	no	`dev`	Informational label (`dev` / `test` / `ci` / `prod`).
`DATABASE_URL`	no (dev default)	`sqlite:///apps/api/chartnav.db`	SQLAlchemy URL. Supports `sqlite:///...` and `postgresql+psycopg://...` .
`CHARTNAV_AUTH_MODE`	no	`header`	`header` (dev only) or `bearer` (prod placeholder).
`CHARTNAV_JWT_ISSUER`	**yes iff** `AUTH_MODE=bearer`	—	OIDC issuer URL.
`CHARTNAV_JWT_AUDIENCE`	**yes iff** `AUTH_MODE=bearer`	—	Expected `aud` claim.
`CHARTNAV_JWT_JWKS_URL`	**yes iff** `AUTH_MODE=bearer`	—	JWKS endpoint for signing-key lookup.
`CHARTNAV_RUN_SEED`	no	`0`	Entrypoint: run `scripts_seed.py` after migrations. Keep `0` in prod.
`API_HOST` / `API_PORT`	no	`0.0.0.0` / `8000`	Uvicorn bind.

Validation on startup

`app.config._load()` is evaluated at import time. It enforces:

- `CHARTNAV_AUTH_MODE ∈ {header, bearer}` .
- If `AUTH_MODE=bearer` , **all three** JWT vars must be set — otherwise

a `RuntimeError` is raised listing the missing names. The app will

not boot half-configured.

Dev / CI / Prod profiles

Profile	How config is supplied	DB	Auth mode	Seed
Dev	`apps/api/.env.example` → `.env` ; Makefile targets	`sqlite:///apps/api/chartnav.db`	`header`	ad-hoc (`make seed`)
Tests	`apps/api/tests/conftest.py` sets env per-test	per-test SQLite in `tmp_path`	`header`	via `scripts_seed.main()`
CI	`.github/workflows/ci.yml` inline env + services	SQLite + live Postgres service	`header`	seed + re-seed in workflow
Prod	platform env / secret store	`postgresql+psycopg://...` via `infra/docker/docker-compose.prod.yml`	`bearer` (when JWT validation is wired)	off by default

Example files

- `apps/api/.env.example` — full dev contract with comments.
- `infra/docker/docker-compose.prod.yml` — wires the API container to

a Postgres service using these env vars; each has a documented

default so the file is runnable as-is for local parity tests.

Database URL specifics

- **SQLite:** `sqlite:///relative/or/absolute/path.db` (three slashes +

absolute path, or three slashes + relative path).

- **Postgres:** `postgresql+psycopg://user:pass@host:5432/dbname` .

The `+psycopg` driver selector is important — SQLAlchemy defaults to

`psycopg2` which is not installed. `psycopg[binary]` is declared in

`apps/api/pyproject.toml` as the `[postgres]` extra.

What this phase explicitly does NOT do

- No secret store integration (AWS SM, Vault, etc.).
- No `.env` auto-loading in the API (docker-compose and operators load env).
- No per-request feature flags / remote config.

Those are safe to layer on top of this contract without reshaping it.

Deploy Target

Chosen path

Docker image built from `apps/api/Dockerfile`, wired to a Postgres service via `infra/docker/docker-compose.prod.yml`. That's the single supported deploy surface in this phase — no Kubernetes, no Terraform, no cloud-specific glue yet.

Hardened Dockerfile

`apps/api/Dockerfile`:

- `python:3.11-slim` base.
- `PYTHONDONTWRITEBYTECODE`, `PYTHONUNBUFFERED`, disabled pip cache.
- Installs `curl` only (for the container healthcheck).
- Installs the API with the `[postgres]` extra (`psycopg[binary]`).
- Copies `app/`, `alembic/`, `alembic.ini`, `scripts/`, `scripts_seed.py`, `entrypoint.sh`.
- Creates a non-root `chartnav` user (uid 1001) and runs as that user.
- `HEALTHCHECK` hits `/health` every 15s with 3 retries.
- `ENTRYPOINT` is `./entrypoint.sh`; `CMD` is the uvicorn invocation.

Entrypoint

`apps/api/entrypoint.sh`:

1. Asserts `DATABASE_URL` is set (fails loudly otherwise).
2. `alembic upgrade head` — migrations always run at start-up.
3. If `CHARTNAV_RUN_SEED=1`, runs `python scripts_seed.py` (idempotent; off by default in prod).
4. `exec "$@"` — hands off to `CMD` so uvicorn becomes PID 1's child.

Migration-on-start is the right default for this app today. It is idempotent, small, and the seed step is gated off in production.

docker-compose.prod.yml

`infra/docker/docker-compose.prod.yml` brings up two services:

- `db` — `postgres:16-alpine` with a persistent volume, healthcheck,

and dev-safe default credentials (operators must override for real

deploys).

- `api` — built from `apps/api/Dockerfile`, depends on `db` with

`condition: service_healthy`, reads all runtime config from env.

All sensitive values (`POSTGRES_PASSWORD`, JWT settings, etc.) are

`${VAR:-default}` so the file is runnable locally for parity tests but

ships nothing secret.

Startup path

```
# clone + cd
cd infra/docker
# optional: override defaults via env
docker compose -f docker-compose.prod.yml up --build
```

Boot sequence per container:

1. Postgres starts, becomes healthy (~5s).

2. API container starts, entrypoint runs migrations.

3. uvicorn binds 0.0.0.0:8000.

4. HEALTHCHECK turns green.

Runtime config path

Every variable is documented in `12-runtime-config.md`. The

image respects only those — there's no bespoke `config.yaml`,

Consul, etc. to reason about.

CI build verification

`.github/workflows/ci.yml` → `docker-build` job:

1. Buildx sets up.

2. Builds `chartnav-api:ci` from `apps/api/`.

3. Runs the container with `DATABASE_URL=sqlite:///./chartnav.db` and

`CHARTNAV_RUN_SEED=1`.

4. Polls `/health`.

5. Runs the same `scripts/smoke.sh` the backend-sqlite job uses.

6. Tears down.

If the image can't boot, serve `/health`, or pass smoke, the job fails.

Out of scope this phase

- Image registry push (no target credentials by default).
- Kubernetes manifests, Helm charts, Terraform.
- TLS termination — expected to be handled by a reverse proxy in front.
- Secret management integration (AWS SM, Vault, etc.).
- Blue/green or rolling deployment orchestration.

Those are the next "hosted deploy" phase's job. The Docker seam is

real and ready to be plugged into any of them.

14 — Postgres parity

Postgres Parity

Why this phase

The app was born on SQLite. This phase proves the same code runs on

Postgres — migrations, seed, auth, scoping, RBAC, state machine.

Code-level changes that made this possible

The DB layer was rewritten to use **SQLAlchemy Core** (`apps/api/app/db.py`).

That single switch buys cross-dialect behavior:

- All SQL uses **named** bind parameters (`:name`). No more `?` .
- Portable functions only — `COALESCE` replaced `IFNULL` ; `CURRENT_TIMESTAMP` already worked on both.
- `insert_returning_id(conn, table, values)` helper encapsulates the

dialect split: `RETURNING id` on Postgres, `cursor.lastrowid` on

SQLite. Callers never touch either.

- `transaction()` context manager gives a SA connection with `begin()`

semantics on either backend.

Consumers changed:

- `apps/api/app/api/routes.py` — all queries now use `:name` binds

and `transaction()` .

- `apps/api/scripts_seed.py` — rewritten to `transaction()` +

`insert_returning_id` , no more raw `sqlite3` .

- `apps/api/app/auth.py` — uses `fetch_one` from the new `db` module.
- `apps/api/alembic/env.py` — honors both `-x sqlalchemy.url=` and

`DATABASE_URL` env, so migrations can run against either backend.

No existing migration (`43ccbf363a8f` , `a1b2c3d4e5f6`) needed changes.

They already used SQLAlchemy constructs and portable defaults.

Proof — `scripts/pg_verify.sh`

Single reproducible command. Launches `postgres:16-alpine` in a

throwaway container, runs migrations + seed, boots the API against it,

runs the full smoke, then does a live **status transition** + reads

the resulting `workflow_events` row back.

Local run on 2026-04-18:

```
==> alembic upgrade head on Postgres
INFO  [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO  [alembic.runtime.migration] Running upgrade  -> 43ccbf363a8f, create initial chartnav tables
INFO  [alembic.runtime.migration] Running upgrade 43ccbf363a8f -> a1b2c3d4e5f6, add encounters and wor

==> seed (idempotent, run twice)
Seed complete.
  demo-eye-clinic: organization_id=1 location_id=1
  northside-retina: organization_id=2 location_id=2

==> smoke: http://127.0.0.1:8766
  ok  /health 200
  ok  /health body.status=ok
  ok  /me 401 without auth
  ok  /me 200 role=admin org=1
  ok  /encounters 401 without auth
  ok  /encounters 200 admin1
  ok  /encounters?organization_id=2 (cross-org) -> 403
  ok  GET /encounters/1 as admin1 -> 200
  ok  GET /encounters/3 cross-org -> 404
==> all smoke checks passed
==> extra: status transition on Postgres
  ok  PT-1001 in_progress -> draft_ready (clinician)
  ok  workflow_events row recorded on Postgres
==> Postgres parity: PASS
```

CI coverage

`.github/workflows/ci.yml` → `backend-postgres` job:

1. Spins up a `postgres:16-alpine` service container on port 5432.

2. Installs the API with the `[postgres]` extra.

3. `alembic upgrade head` (direct env `DATABASE_URL=postgresql+psycopg://…`).

4. Seeds twice (idempotency proof).

5. Boots uvicorn, waits for `/health` .

6. Runs `scripts/smoke.sh` .

7. Executes the same `clinician → draft_ready` status transition and

asserts the returned row.

If any step fails, the job fails and nothing merges.

What's covered / not covered

Surface	Verified on Postgres
Schema migrations	✅
Idempotent seed	✅
App boot	✅
`/health`, `/me`	✅
`/encounters` (list + filters + cross-org 403)	✅
`/encounters/{id}` 200 / 404 cross-org	✅
State transition write + event emit	✅
pytest suite (28 tests)	Runs on SQLite only today — see Known Gaps.

The pytest suite still targets SQLite because the TestClient fixture uses `tmp_path` file DBs. Migrating that to a Postgres matrix is the obvious next CI upgrade; the DB layer is ready for it.

Diagram — System architecture

System Architecture

```
flowchart LR subgraph Clients C["Client
curl · tests · apps/web"] end subgraph Runtime FastAPI["FastAPI
apps/api/app/main.py"] Config["settings
apps/api/app/config.py
(env-driven)"] Authn["require_caller
auth.py
header · bearer"] Authz["authz.py
RBAC"] Router["APIRouter
routes.py"] SM["State Machine
ALLOWED_TRANSITIONS"] DB["db.py
SQLAlchemy Core"] Engine[(SA Engine)] end subgraph Storage SQLite[("SQLite
apps/api/chartnav.db")] PG[("Postgres 16")] end subgraph Deploy Dockerfile["Dockerfile
(non-root, healthcheck)"] Entry["entrypoint.sh
migrate + exec"] Compose["infra/docker/
docker-compose.prod.yml"] end subgraph Tooling Alembic["Alembic
DATABASE_URL-aware"] Seed["scripts_seed.py
(cross-dialect)"] PyTest["pytest (28)"] Smoke["scripts/smoke.sh"] Verify["scripts/verify.sh"]
PgVerify["scripts/pg_verify.sh"] DocBuild["scripts/build_docs.py"] Make["Makefile"] CI[".github/workflows/ci.yml
backend-sqlite · backend-postgres · docker-build · docs"] end C -->|"HTTP + X-User-Email or Bearer"| FastAPI
FastAPI --> Authn Authn --> Authz Authz --> Router Router --> SM Router --> DB DB --> Engine Engine --
>|"sqlite:///..."| SQLite Engine -->|"postgresql+psycopg://..."| PG Config -.-> Authn Config -.-> DB Alembic -.->
Engine Seed -.-> DB PyTest -.->|"TestClient"| FastAPI Smoke -.->|"curl"| FastAPI Verify --> Make PgVerify --> Make
DocBuild --> Make Make --> Alembic Make --> Seed Make --> PyTest CI --> Make Dockerfile --> Entry Entry -->
Alembic Entry --> FastAPI Compose --> Dockerfile Compose --> PG
```

Diagram — Encounter status machine

Encounter Status State Machine

```
stateDiagram-v2 [*] --> scheduled [*] --> in_progress : walk-in scheduled --> in_progress in_progress --> draft_ready draft_ready --> review_needed draft_ready --> in_progress : rework review_needed --> completed review_needed --> draft_ready : kick back completed --> [*]
```

Legend: any edge not shown is rejected at `POST /encounters/{id}/status`

with HTTP 400 `invalid_transition`.

Diagram — ER

ER Diagram

erDiagram organizations ||--o{ locations : "has" organizations ||--o{ users : "employs (identity + role)" organizations ||--o{ encounters : "owns" locations ||--o{ encounters : "hosts" encounters ||--o{ workflow_events : "emits" organizations { int id PK string name string slug UK datetime created_at } locations { int id PK int organization_id FK string name datetime created_at } users { int id PK int organization_id FK string email UK string full_name string role "admin | clinician | reviewer" datetime created_at } encounters { int id PK int organization_id FK int location_id FK string patient_identifier string patient_name string provider_name string status datetime scheduled_at datetime started_at datetime completed_at datetime created_at } workflow_events { int id PK int encounter_id FK string event_type text event_data datetime created_at }

Seeded tenants & users

org_id	slug	email	role
1	`demo-eye-clinic`	admin@chartnav.local	admin
1	`demo-eye-clinic`	clin@chartnav.local	clinician
1	`demo-eye-clinic`	rev@chartnav.local	reviewer
2	`northside-retina`	admin@northside.local	admin
2	`northside-retina`	clin@northside.local	clinician

`users.email` is the authentication key consumed from `X-User-Email`.

`users.role` is the RBAC key consumed by `app.authz`.

`users.organization_id` is the authoritative source of scope; never

derived from client input.

Diagram — API / data flow

API / Data Flow & CI

Request path

```
sequenceDiagram autonumber participant C as Client participant Cfg as app.config participant Authn as require_caller participant Authz as authz gate participant R as Route handler participant SM as State Machine participant DB as SQLAlchemy Core → SQLite / Postgres Note over C,DB: header mode happy path C->>Authn: POST /encounters/1/status {"status":"draft_ready"}
X-User-Email: clin@chartnav.local Cfg-->>Authn: auth_mode=header Authn->>DB: SELECT user WHERE email=:email DB-->>Authn: {role=clinician, org=1} Authn-->>Authz: Caller Authz->>Authz: role in CAN_CREATE / TRANSITION? Authz-->>R: Caller R->>DB: SELECT encounter 1 ... DB-->>R: row(org=1, status=in_progress) R->>SM: (in_progress → draft_ready) allowed? SM-->>R: yes R->>Authz: assert_can_transition(clinician, ...) Authz-->>R: allowed R->>DB: UPDATE + INSERT status_changed{changed_by} R-->>C: 200 Note over C,DB: bearer mode honest refusal C->>Authn: GET /me
Authorization: Bearer ... Cfg-->>Authn: auth_mode=bearer Authn-->>C: 501 auth_bearer_not_implemented Note over C,DB: cross-org read C->>Authn: GET /encounters/3 as org1 R->>DB: SELECT encounter 3 DB-->>R: row(org=2) R-->>C: 404 encounter_not_found Note over C,DB: bad bearer config at boot Cfg->>Cfg: CHARTNAV_AUTH_MODE=bearer but JWT env missing Cfg-->>C: RuntimeError — app refuses to import
```

CI gate flow

```
flowchart TD PR[["push / PR"]] --> SQL[backend-sqlite] SQL --> PG[backend-postgres] SQL --> DK[docker-build] SQL --> DOC[docs] subgraph "backend-sqlite" A1[install deps] --> A2[alembic upgrade] A2 --> A3[seed x2] A3 --> A4[pytest 28] A4 --> A5[scripts/verify.sh] end subgraph "backend-postgres (service: postgres:16-alpine)" B1[install + psycopg] --> B2[alembic upgrade] B2 --> B3[seed x2] B3 --> B4[boot + scripts/smoke.sh] B4 --> B5[live status transition] end subgraph "docker-build" D1[buildx build] --> D2[run container] D2 --> D3[scripts/smoke.sh] end subgraph "docs" X1[apt chromium] --> X2[scripts/build_docs.py] X2 --> X3[upload artifact] end
```

Any red-bordered step failing fails CI.